

**CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
CAMPUS TIMÓTEO**

João Pedro Ferreira Duarte

**GERAÇÃO TEXT-TO-SQL VIA EXPLORAÇÃO AGENTICA DE
METADADOS: UMA ARQUITETURA PARA REDUÇÃO DE
ALUCINAÇÕES ESTRUTURAIS EM LLMS**

Timóteo

2026

João Pedro Ferreira Duarte

**GERAÇÃO TEXT-TO-SQL VIA EXPLORAÇÃO AGENTICA DE
METADADOS: UMA ARQUITETURA PARA REDUÇÃO DE
ALUCINAÇÕES ESTRUTURAIS EM LLMS**

Monografia apresentada à Coordenação de Engenharia de Computação do Campus Timóteo do Centro Federal de Educação Tecnológica de Minas Gerais para obtenção do grau de Bacharel em Engenharia de Computação.

Orientador: Aléssio Miranda Júnior
Coorientador: Evandrino Gomes Barros

Timóteo

2026

Resumo

Este trabalho propõe e avalia um pipeline *Text-to-SQL* orientado por metadados, no qual um agente LLM consulta ferramentas MCP antes da geração de consultas. O objetivo é reduzir alucinações estruturais e aumentar a executabilidade do SQL em comparação com abordagens baseadas apenas em *schema* estático no *prompt*. O artefato integra Spring AI, servidor MCP e Apache Atlas como fonte canônica de metadados, com avaliação experimental em ambiente controlado por métricas de aderência estrutural, executabilidade, rastreabilidade e custo financeiro da operação do pipeline por consulta.

Palavras-chave: *Text-to-SQL*; Model Context Protocol; metadados; Apache Atlas; agentes LLM.

Abstract

This work proposes and evaluates a metadata-grounded *Text-to-SQL* pipeline in which an LLM agent consults MCP tools before SQL generation. The main goal is to reduce structural hallucinations and improve SQL executability compared to prompt-only approaches based on static schema context. The artifact integrates Spring AI, an MCP server, and Apache Atlas as the canonical metadata source, and is evaluated in a controlled setup using structural adherence, executability, traceability, and financial cost of the pipeline operation per query metrics.

Keywords: *Text-to-SQL*; Model Context Protocol; metadata; Apache Atlas; LLM agents.

Lista de ilustrações

Figura 1 – Fluxo lógico do artefato alvo, 2026.	9
Figura 2 – Hierarquia do paradigma de inteligência artificial (adaptado de (RASCHKA, 2024)).	12
Figura 3 – Fluxos metodológicos consolidados: preparação da massa de avaliação e ciclo experimental do artefato.	22

Lista de tabelas

Tabela 1 – Bateria piloto de perguntas por cenário metodológico.	21
Tabela 2 – Cronograma resumido por mês (modelo inicial; jul.–nov./2026).	23

Lista de abreviaturas e siglas

AI	Artificial Intelligence (Inteligência Artificial)
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
BIRD	Big Bench for Large-Scale Database
ETL	Extract, Transform, Load (Extração, transformação e carga)
JSONL	JSON Lines
LLM	Large Language Model
MCP	Model Context Protocol
NLI-DB	Natural Language Interface to Database (Interface em linguagem natural para bases de dados)
NL	Natural Language (Linguagem Natural)
POMDP	Partially Observable Markov Decision Process
RAG	Retrieval-Augmented Generation
SQL	Structured Query Language

Sumário

1	INTRODUÇÃO	8
1.1	Contexto e motivação	8
1.2	Problema de pesquisa	8
1.3	Justificativa	8
1.4	Objetivos	9
2	FUNDAMENTAÇÃO TEÓRICA	10
2.1	Interfaces em Linguagem Natural e Bases de Dados	10
2.2	Text-to-SQL	11
2.3	Modelos de Linguagem e Mediação Instrumental	12
2.4	Catálogos, pipelines centrados em dados e <i>grounding</i>	13
2.4.1	Contexto de dados e pré-processamento	14
2.4.2	Grounding e metadados de catálogo	14
2.5	Catálogo no Contexto Experimental	15
2.6	Model Context Protocol	15
2.7	Confiabilidade da Síntese	15
2.8	Síntese	15
3	TRABALHOS RELACIONADOS	16
4	PROCEDIMENTOS METODOLÓGICOS	17
4.1	Ambiente, materiais e dados	17
4.2	Métodos	18
4.2.1	Unidade experimental e vocabulário	18
4.2.2	Arquitetura do artefato	19
4.2.3	Procedimento de implementação	19
4.2.4	Protocolo de avaliação	20
4.2.5	Registro experimental e ameaças à validade	21
5	CRONOGRAMA DE EXECUÇÃO	23
5.1	Cronograma de execução	23
5.1.1	Tabela mês × atividade	23
	REFERÊNCIAS	24

1 Introdução

1.1 Contexto e motivação

Com o avanço de técnicas de *Machine Learning*, modelos *Large Language Model* (LLM) têm sido cada vez mais aplicados no contexto de tradução de linguagem natural para linguagens formais declarativas e parcialmente sensíveis ao contexto como o SQL. Neste sentido, técnicas de codificação-decodificação (*Seq2Seq*) evoluem para aplicarem as técnicas de aprendizado de máquina para otimização das expressões formais criadas à partir de linguagem natural.

1.2 Problema de pesquisa

Em consequência do surgimento de tecnologias de *Deep Learning* e consequente consumo por parte não técnica da população, interfaces de *Text-to-SQL* ganharam notoriedade no cenário de aplicações onde é possível realizar consultas de variada complexidade em bases de dados sem prévio conhecimento de SQL (LIU et al., 2026). Entretanto, apesar do significativo avanço dessa tecnologia, ainda existem gargalos no processo de tradução dessas linguagens de diferentes domínios. Isso posto, as duas principais causas de problemas de tradução, são, primeiramente, a ambiguidade semântica de qualquer linguagem natural e a consequente baixa interpretabilidade dos modelos quando expostos à expressões complexas, como também, as restrições no domínio semântico do SQL, quando da tentativa de replicar em linguagem regular, as operações desejadas especificadas na linguagem de origem.

O problema investigado consiste em como aumentar a aderência estrutural e a executabilidade de consultas SQL geradas por LLMs em ambiente experimental, mantendo rastreabilidade do processo de decisão e manutenção da adequação ao contexto de origem. Em especial, busca-se reduzir alucinações estruturais como referência a tabelas, colunas ou relacionamentos inexistentes no catálogo canônico de metadados.

1.3 Justificativa

Este trabalho se justifica pela necessidade de, através da implementação e estudo de uma arquitetura *Text-to-SQL*, que implementa tecnologia de aprendizado de máquina, tornar a barreira técnica de realização de consultas SQL em bases de dados, por parte não técnica da população, cada vez menor, como também, impulsionar a capacidade de utilização dessas bases de dados, no contexto de melhorar a qualidade e confiabilidade de decisões tomadas conforme insights extraídos deste contexto.

No presente documento, também é possível ver, além de uma proposta arquitetônica da implementação do artefato alvo, também um estudo dirigido em caso de uso, tal qual ao

avaliar de forma crítica o desempenho do artefato, considerando as métricas de análise de alucinações estruturais e executabilidade de consultas SQL.

1.4 Objetivos

O objetivo de execução do presente projeto, tem como alvo construir e avaliar um artefato *Text-to-SQL* mediado por servidor MCP e apoiado por catálogo de metadados vinculado a um banco de dados relacional, com foco na redução de alucinações estruturais e no aumento da confiabilidade da geração de sentenças SQL executáveis conforme ilustrado na Figura 1.

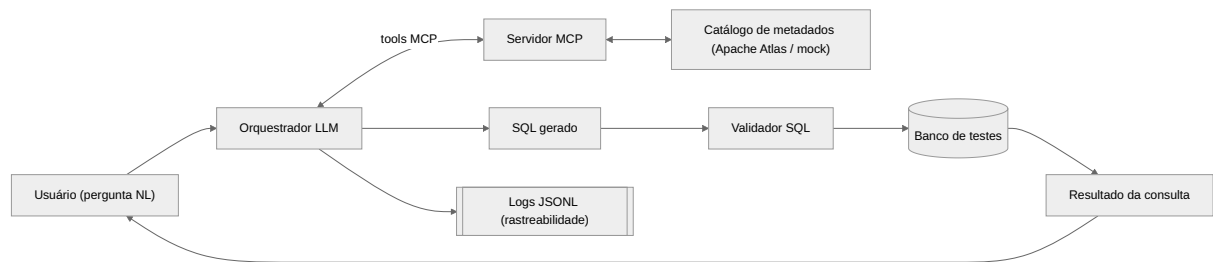


Figura 1 – Fluxo lógico do artefato alvo, 2026.

São definidos como objetivos específicos deste trabalho, os que se seguem:

1. Projetar a arquitetura do artefato com orquestrador LLM, servidor MCP e adaptador de catálogo;
2. Definir e implementar *tools* MCP para descoberta de tabelas, colunas e relacionamentos;
3. Avaliar o artefato em ambiente controlado com métricas de aderência estrutural, executabilidade, rastreabilidade e custo de chamadas de ferramenta;

2 Fundamentação Teórica

2.1 Interfaces em Linguagem Natural e Bases de Dados

Interfaces em NL para bases de dados (*Natural Language Interfaces to Databases*) tratam do espaço de desenho em que intenções formuladas em NL devem ser mapeadas para operações consultivas sobre repositórios estruturados (LIU et al., 2026). Do ponto de vista conceitual, há uma assimetria fundamental: a NL é tolerante a ambiguidade e omissão, enquanto o modelo relacional exige predicados explícitos sobre um esquema declarado e um vocabulário de símbolos estáveis. Neste contexto, assumimos essa mediação como problema central de representação, anteriormente a quaisquer definições e escolhas arquiteturais ou metodológicas.

Segundo (ANDROUTSOPOULOS; RITCHIE; THANISCH, 1995), no final da década de 1960 já existiam protótipos que propunham NLI-DB para a interface de bases de dados em NL. Nessa época, o modelo mais conhecido proposto foi LUNAR que era usado para análise de composição química em pedras lunares. Assim como para outras propostas, mesmo que houvesse modelagem interna para generalizar o domínio semântico da aplicação, o uso ainda era muito vinculado ao domínio de negócio dos dados persistidos.

Nas décadas seguintes do século XX, modelos propunham contextos mais avançados, como estabelecimento de perguntas ao usuário antes da criação da consulta, intercalação de processamento sintático e semântico, transformação de NL em expressões lógicas de linguagens declarativas, como já foi feito com Prolog e ensinar o sistema novas palavras durante a interação, assim como fazia o Ask.

Neste contexto, em função da evolução do paradigma, modelos metodológicos foram propostos com o objetivo de resolver o mesmo problema. Uma síntese recente (LIU et al., 2026) permite organizá-los da seguinte forma:

1. **Regras:** Árvore de análise sintática, ontologia, grafo semântico.
2. **Aprendizado de máquina:** Redes neurais, aprendizado de máquina tradicional.
3. **Modelos de linguagem pré-treinados:** BERT-based, modelos baseados em grafo, *template filling*, métodos generativos puros.
4. **LLM:** engenharia de *prompt*, *fine-tuning*, estrutura integrada de conhecimento de base de dados.

Dessa forma, os modelos de interface entre NL e linguagem formal evoluíram de forma a aplicar conceitos de *Seq2Seq* (SUTSKEVER; VINYALS; LE, 2014) para realizar a geração direta das expressões formais criadas a partir de NL.

A arquitetura *sequence-to-sequence* generalizou a ideia de mapear um enunciado em NL para um programa simbólico de comprimento variável, abrindo caminho a modelos neurais de análise semântica em que a atenção alinha fragmentos da pergunta aos constituintes da linguagem alvo (BAHDANAU; CHO; BENGIO, 2014). Com o predomínio de representações contextualizadas pré-treinadas e, mais recentemente, de modelos generativos de grande escala, o mesmo enquadramento passou a ser explorado sob pressões de generalização a esquemas e consultas inéditos, consolidando a tarefa de *Text-to-SQL* como caso paradigmático de interface consultiva (YU et al., 2018; LIU et al., 2026). O que se segue caracteriza essa tarefa e as exigências de *schema linking* e de composição sintática de SQL que dela decorrem.

2.2 Text-to-SQL

Text-to-SQL nomeia a tarefa de produzir uma consulta em SQL, semanticamente compatível com um esquema disponível, a partir de uma pergunta em NL (LIU et al., 2026). Ela herda a tensão referida acima: é preciso resolver ambiguidades da pergunta, relacionar termos do usuário a nomes de relações e atributos (*schema linking*) e compor uma formulação executável no dialeto pretendido. Benchmarks clássicos formulam a tarefa em regimes em que esquemas ou consultas não reaparecem integralmente entre treino e teste, explicitando a exigência de generalização (YU et al., 2018).

As configurações posteriores de grande escala continuam a evidenciar dependência entre pergunta, evidência de esquema ou dados e o comportamento observado de sistemas centrados em modelos de linguagem (LI et al., 2023). A tarefa de *Text-to-SQL* evoluiu de uma forma controlada, com técnicas baseadas em regras, para uma forma flexível, com paradigma centralizado em NL. Essa mudança leva a *trade-offs* entre acurácia, interpretabilidade, generalização, eficiência de custo e transferibilidade de domínio. Para reduzir a lacuna entre os estudos acadêmicos e as aplicações do mundo real, propostas mais modernas como o *benchmark BIRD* (LI et al., 2023) realocaram o foco da mera compreensão estrutural do esquema para a complexidade dos valores reais armazenados. Esse novo formato evidencia que ainda há desafios que permanecem em aberto, notadamente: a dificuldade de lidar com bases de dados massivas que contêm valores ruidosos; a necessidade de raciocínio baseado em conhecimento externo para alinhar a intenção do usuário aos dados de domínio; e a exigência por eficiência de execução (*execution efficiency*) das consultas geradas. Conforme demonstrado pelas avaliações do BIRD, a acurácia dos modelos disponíveis em 2023 (como o GPT-4) ainda ficava muito aquém da performance humana (cerca de 54,89% contra 92,96%), mostrando que a compreensão profunda dos valores do banco de dados para a geração de consultas semanticamente e logicamente corretas continuava a ser um fator de desenvolvimento (LI et al., 2023).

O método DIN-SQL (POURREZA; RAFIEI, 2023) exemplifica uma resposta metodológica a essa lacuna: decompor a geração em subproblemas sequenciais, reinserir as soluções intermédias no contexto (*decomposed in-context learning*) e acrescentar etapa de autocorreção assistida pelo próprio LLM, mitigando a dificuldade de gerar SQL declarativo de uma só vez a partir da pergunta. Em experimentos reportados pelos autores, o ganho face a *few-shot*

simples situa-se da ordem de dez pontos percentuais nos modelos considerados; no conjunto de teste *holdout* do Spider, com GPT-4, alcança-se 85,3% de *execution accuracy*, superando o melhor resultado então citado para a mesma métrica (79,9%); no BIRD, obtém-se 55,9% de precisão de execução no *holdout*, então apresentado como estado da arte nesse *benchmark* de 2023. Essa combinação de decomposição, evidência incremental e revisão antecipa a linha de trabalhos subsequentes que revisitam Spider e BIRD com arquiteturas centradas em LLM, como se verifica a seguir.

Em contexto ainda mais atual, trabalhos como (WANG et al., 2025; JIAN et al., 2026) retomam os mesmos benchmarks já estabelecidos, Spider e BIRD, com modelos LLM mais recentes. Nesses estudos, arquiteturas de *Text-to-SQL* centradas em LLM alcançam, em conjunto, faixas reportadas de acurácia de execução da ordem de 85%–88% no conjunto de teste do Spider e de cerca de 65%–77% no BIRD, conforme métricas e partições de avaliação definidas pelos autores.

2.3 Modelos de Linguagem e Mediação Instrumental

Os LLMs são, em síntese operacional, modelos de rede neural profunda, treinados em grandes corpus textuais e estruturados em torno da arquitetura *transformer*, capazes de processar e gerar sequências que parecem coerentes e contextualmente adequadas, sem que tal “adequação” implique consciência ou compreensão humanas (RASCHKA, 2024). No âmbito de interfaces entre NL e bases de dados, trabalhos de enquadramento recente posicionam os LLMs como peça central para aproximar intenções em linguagem natural de programas consultivos em *SQL*, prolongando a mediação já discutida para NLI-DB e para a tarefa de *Text-to-SQL*. Nesse papel de camada de tradução, explora-se a geração condicionada ao contexto observado para produzir formulação simbólica executável sob restrições de esquema, motivando o tratamento que se segue.

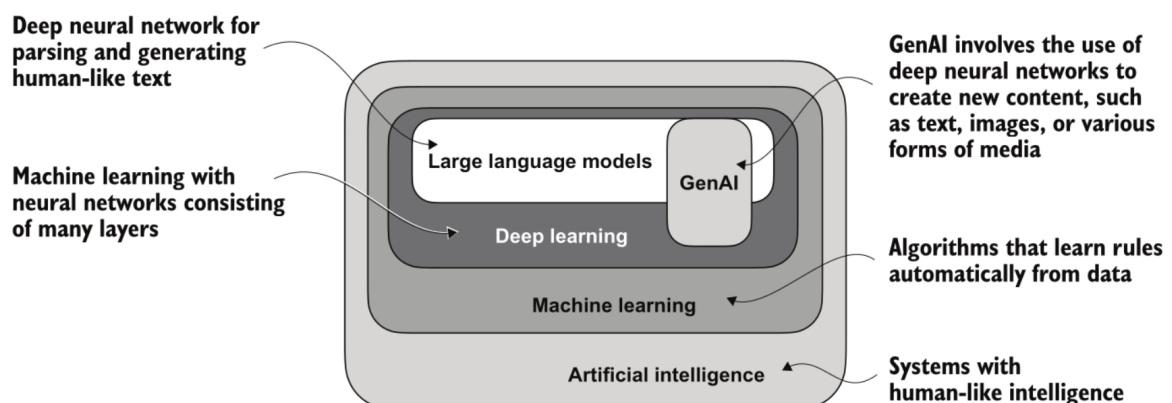


Figura 2 – Hierarquia do paradigma de inteligência artificial (adaptado de (RASCHKA, 2024)).

A Figura 2 demonstra a hierarquia simplificada de inteligência artificial, aprendizado de máquina e *deep learning*, de forma a ilustrar que os LLMs representam uma aplicação específica das técnicas de aprendizado profundo, de forma que todos estes fazem parte de um para-

digma maior de IA. Do ponto de vista arquitetural, a proposta clássica do *transformer* descreve pilhas de codificador e de decodificador interligadas por mecanismos de atenção (VASWANI et al., 2017), ponto de partida para situar como a transdução em sequência sustenta a camada de tradução entre NL e consultas em *SQL* já mencionada.

A arquitetura *transformer* introduzida por (VASWANI et al., 2017) segue o esquema codificador–decodificador usual em modelos neurais de transdução: o codificador mapeia uma sequência de símbolos de entrada para uma sequência de representações contínuas intermédias; dada essa sequência, o decodificador gera a sequência de saída um elemento de cada vez, de modo autorregressivo — em cada passo o modelo condiciona-se aos símbolos de saída já emitidos. Tanto o codificador como o decodificador são pilhas de camadas idênticas repetidas: em cada camada do codificador combinam-se atenção *multi-head self-attention* e uma rede *feed-forward* pontual, com ligações residuais e normalização de camada; em cada camada do decodificador acrescentam-se, além da *self-attention* mascarada e da mesma rede pontual, um módulo de atenção *multi-head* sobre a saída do codificador. O artigo motiva o desenho com tarefas como a tradução entre línguas; a mesma noção de transdução em sequência ilustra, por analogia, o mapeamento de uma pergunta em NL para um programa em *SQL*, sem que o artigo original trate explicitamente de bases de dados.

Convém notar que o *Transformer* original é *encoder–decoder*, ao passo que muitos LLM usados hoje em geração de texto e de código adotam variantes *decoder-only* (linha de modelos do tipo GPT e arquiteturas análogas), isto é, uma pilha causal única, sem pilha de codificador separada como na definição acima (RASCHKA, 2024). Mantém-se, contudo, o núcleo de atenção e a geração token a token condicionada ao contexto observado.

LLMs funcionam, nesse enquadramento de atenção e geração autoregressiva, como mecanismos probabilísticos de geração textual condicionada ao contexto observado. Historicamente, arquiteturas sequenciais com atenção posicionaram-se como referência para alinhar fragmentos da entrada aos símbolos emitidos na saída formal (BAHDANAU; CHO; BENGIO, 2014); em modelos atuais admite-se raciocínio parcelado e recurso a funções externas sempre que o contexto em *prompt* é insuficiente para fixar o programa (LIU et al., 2026). Em *Text-to-SQL*, esse recurso toma a forma de mediação instrumental: o modelo deixa de depender exclusivamente de uma descrição estática do esquema e pode solicitar informação estruturada em passos sucessivos. A mediação instrumental, por si só, não substitui a necessidade de amarrar a geração a evidências auditáveis de esquema e de dados, tema que a seção seguinte aborda sob o prisma de catálogos, de *pipelines* centrados em dados e de *grounding* apoiado em metadados de catálogo.

2.4 Catálogos, pipelines centrados em dados e *grounding*

O avanço da aplicação de recursos de aprendizado de máquina a *pipelines* de tradução entre NL e *SQL*, embora substancial, ainda expõe dificuldades de pré-processamento de requisitos — seja do ponto de vista do *schema*, dos dados ou do *input* recebido. O que se segue organiza esse tema em duas subsecções: primeiro o *contexto de dados* e as etapas de pré-processamento (como referência, **ETL**); em seguida, *grounding* e metadados de catálogo,

onde a literatura costuma situar os ganhos de precisão após evidências de esquema estarem disponíveis ao modelo.

2.4.1 Contexto de dados e pré-processamento

Nesta monografia, **contexto de dados** nomeia, de maneira operacional, o conjunto de informações de domínio, de esquema relacional disponível e de entradas textuais que circulam no *pipeline* antes e durante a geração de *SQL* — isto é, tudo o que o sistema pode inspecionar para reduzir ambiguidades entre a intenção em linguagem natural e os símbolos do banco.

Pré-processamento reúne etapas como saneamento, harmonização de formatos, duplicação parcial, enquadramento de valores ruidosos ao *schema* e preparação de evidências tabulares ou textuais consumidas por modelos ou por serviços auxiliares. Em conjunto, buscam diminuir ruído e desalinhamentos entre a fonte bruta e o uso analítico ou generativo.

Um referencial clássico de orquestração desses fluxos é o *pipeline ETL* (*Extract, Transform, Load*: extração a partir das fontes, transformação segundo regras de negócio e de qualidade, carga em destino analítico ou transacional). Em desenhos com característica de *Data-centric ML Pipeline*, o tratamento da qualidade dos dados e a definição de artefatos reutilizáveis no treino ou na inferência situam-se nessa cadência: antes de se pleitear ganho de acurácia na sentença final, estabiliza-se o que será observado pelo modelo.

Metadados descritivos e de governança — inventário de relações e atributos, tipos, *constraints*, linhagem — são, em geral, *registrados após* (ou em continuidade explícita com) etapas de integração e de padronização dos dados: uma vez que objetos passíveis de descrição estável resultam da extração, da transformação e da carga, o catálogo pode formalizar declarações auditáveis sobre o que “existe” no sistema de informação, em complemento ao volume físico de tuplas. Essa ordem explica por que o enriquecimento por metadados aparece, na prática, como camada posterior ao pré-processamento, e não como substituto dele.

2.4.2 Grounding e metadados de catálogo

No contexto da tentativa de aumentar a acurácia da sentença *SQL* final por meio da aplicação de catálogos de metadados, relatos na literatura alcançam, por exemplo, 91,8% de precisão (FATHOLLAHZADEH; MANSOUR; BOEHM, 2025).

Grounding, no presente uso, designa a disciplina de amarrar decisões de geração a evidências observáveis sobre o domínio dos dados. Metadados de catálogo — nomes, tipos, *constraints*, descrições textuais e relacionamentos — funcionam como uma camada de declarações auditáveis sobre o que “existe” no sistema de informação, distinta do volume de tuplas armazenadas. Quando preservados e consultados de forma explícita, contribuem para reduzir o espaço de hipóteses compatíveis sobre tabelas e caminhos de junção. Estudos de geração de *Text-to-SQL* com enriquecimento explícito por metadados ilustram esse princípio (WANG et al., 2025). Trabalhos sobre geração guiada por catálogo em pipelines centrados em dados — domínio vizinho ao deste TCC — reforçam o papel de governança de metadados como sinal de orientação para modelos generativos (FATHOLLAHZADEH; MANSOUR; BOEHM, 2025).

2.5 Catálogo no Contexto Experimental

O recorte experimental deste trabalho utiliza Apache Atlas como instância de catálogo de metadados e governança acessível por serviços (The Apache Software Foundation, 2025). A escolha é de engenharia: concretiza a noção de fonte canônica de esquema em continuidade ao enquadramento das subsecções anteriores desta secção, relativamente a metadados de catálogo e a *grounding*. Não se atribui, nesta fundamentação, desempenho ou completude de produto além do que resultar do desenho experimental e da documentação oficial citada.

2.6 Model Context Protocol

O *Model Context Protocol* (MCP) descreve, ao nível de integração entre componentes de *software*, um papel cliente–servidor para expor ferramentas, recursos e fluxos coordenados com modelos (ANTHROPIC; Model Context Protocol contributors, 2024). Para a argumentação deste capítulo interessa apenas a abstração: um canal tipado e repetível por onde o agente solicita informação de contexto — neste caso metadados — sem acoplar a semântica da pergunta à implementação interna do catálogo.

2.7 Confiabilidade da Síntese

A confiabilidade da síntese distingue, para os efeitos desta monografia, a coerência estrutural (a consulta referencia apenas artefatos presentes no esquema sob governação) de outras noções mais amplas de correção de negócio. Erros estruturais — por exemplo inventar uma relação ou atributo — constituem falhas falsificáveis relativamente ao catálogo, independentemente de o resultado da consulta parecer “plausível” em NL. Há trabalhos que classificam e procuram detectar esse tipo de desvio mediante critérios de verificação que não presumem apenas um resultado tabular gabarito, incluindo linhas metamórficas aplicáveis ao *Text-to-SQL* assistido por LLM (YANG et al., 2025). O desenvolvimento metodológico concreto do presente relatório será consistente com essa diferenciação.

2.8 Síntese

Em conjunto, os conceitos acima compõem a base teórica mínima para o desenvolvimento do artefato: uma tarefa *Text-to-SQL* exposta a ambiguidade e a restrições de esquema; modelos de linguagem cuja exatidão estrutural melhora quando a inferência é mediada por evidência obtida fora do *prompt* estático; metadados de catálogo como instrumento de *grounding*; exigência de confiabilidade da síntese definida em termos de aderência ao esquema; e um protocolo de integração que formaliza o acesso a essa evidência. O capítulo seguinte situa o estado da arte e os trabalhos que instanciam essas dimensões com abordagens nomeadas e comparáveis.

3 Trabalhos Relacionados

Os trabalhos correlatos foram selecionados pelo alinhamento com três eixos: (i) geração *Text-to-SQL* em esquemas complexos, (ii) uso de metadados e ferramentas para suporte à geração de consultas e (iii) robustez, validação e detecção de falhas em SQL gerado por LLMs.

No primeiro eixo, destacam-se abordagens que enriquecem o processo de *schema linking* com metadados estruturados, como em MCI-SQL (WANG et al., 2025), buscando maior acurácia e consistência na geração de consultas. Esse grupo é especialmente relevante para este TCC por tratar diretamente do papel do contexto de catálogo na qualidade da saída SQL.

No segundo eixo, trabalhos como TRUST-SQL (JIAN et al., 2026) exploram cenários em que o esquema não é integralmente fornecido no *prompt*, exigindo exploração progressiva por chamadas de ferramenta em múltiplos turnos. Esse paradigma aproxima-se da proposta deste projeto, que usa servidor MCP para consulta dirigida de metadados antes da síntese final do SQL.

No terceiro eixo, o trabalho de Yang et al. (YANG et al., 2025) discute mecanismos de detecção de alucinações e falhas estruturais na geração *Text-to-SQL* assistida por LLM, mediante teste metamórfico em duas fases, sem depender exclusivamente de respostas gabarito. Esse recorte contribui para a definição de métricas e critérios de avaliação voltados à confiabilidade do pipeline.

Em síntese, a literatura recente confirma ganhos com uso de metadados e ferramentas, porém ainda há lacunas em protocolos reproduzíveis de avaliação em recortes de artefato único com catálogo canônico e trilha auditável de decisões. Este trabalho se posiciona nessa lacuna ao integrar MCP e Atlas em um pipeline experimental mensurável por aderência estrutural, executabilidade, rastreabilidade e custo de interação.

4 Procedimentos Metodológicos

A presente pesquisa possui natureza aplicada, exploratória e experimental no contexto dos objetivos definidos no Capítulo de Introdução. Adota-se como objeto de estudo um único artefato *Text-to-SQL* mediado por *Model Context Protocol* (MCP), no qual a descoberta de metadados antecede a geração de SQL. O interesse metodológico central reside na avaliação sistemática da aderência estrutural, da executabilidade e da rastreabilidade da cadeia decisória do artefato em ambiente laboratorial reproduzível (FATHOLLAHZADEH; MANSOUR; BOEHM, 2025; LI et al., 2023).

4.1 Ambiente, materiais e dados

O ambiente metodológico da pesquisa é organizado por papéis funcionais e fluxos lógicos do objetivado artefato final. Para garantir rastreabilidade do experimento, os materiais são agrupados por função no ciclo de execução.

- **Software:** Java 25, Spring Boot, Spring AI e servidor MCP.
- **Catálogo e dados:** Apache Atlas e massa relacional XPTO (subconjunto congelado para avaliação).
- **Execução e evidências:** validador e executor SQL controlado, além de *harness* experimental para trilha de sessão.
- **Modelos de linguagem:** provedores OpenAI e Google, com *modelVersion* fixada por corrida.
- **Ambiente laboratorial:** nuvem pública reproduzível, com detalhamento técnico no Capítulo de Desenvolvimento.

A preparação da massa prioriza encadeamento auditável desde a extração até a disponibilização para sessões experimentais, conforme o painel (a) da Figura 3 ao final deste capítulo.

No plano de dados, adota-se como fonte relacional o domínio XPTO, com subconjunto congelado de tabelas representativas dos cenários de avaliação (franquias, projetos, transações, produtos e geografia). A massa de avaliação consiste em subconjunto anonimizado derivado da base operacional da empresa Putz Studio. Esse recorte permite executar consultas simples, junções, filtros temporais e agregações em um corpus coerente com o problema investigado. O dicionário físico completo permanece como referência documental, enquanto o subconjunto efetivamente exposto ao agente é governado pelo catálogo canônico.

A estratégia de escala considera duas ondas metodológicas complementares. Na primeira, utiliza-se a massa congelada de referência para estabilizar o protocolo e verificar con-

sistência do artefato em condições controladas. Na segunda, realiza-se amplificação sintética da massa para estudo de robustez em maior volume, preservando integridade referencial e versionamento explícito de dados. Dessa forma, evita-se de trabalhar o experimento numa massa de dados pequena, de forma que ainda possamos explorar casos em que o volume dos dados possa ser causa suficiente, ou parte da causa, para algum tipo de alucinação ou erro de execução.

Por fim, os provedores de modelo considerados nas corridas são OpenAI e Google, com fixação de *model/Version* por execução. A decisão por múltiplos provedores busca reduzir risco de conclusões excessivamente dependentes de um único comportamento de inferência, mantendo rastreabilidade de configuração em cada corrida.

4.2 Métodos

Os métodos adotados estruturam-se em quatro frentes: arquitetura do artefato, procedimento de implementação, protocolo de avaliação e governança de registro/validade. O princípio comum entre essas frentes é a auditabilidade de cada corrida experimental, da pergunta inicial ao desfecho classificado.

4.2.1 Unidade experimental e vocabulário

Para execução do procedimento, as convenções abaixo são adotadas: cada item da bateria de perguntas gera **uma** pergunta, **uma** sessão e **uma** corrida (relação 1:1:1).

- **Pergunta:** enunciado em linguagem natural submetido ao artefato *Text-to-SQL*, com formulação fixada na bateria de avaliação (por exemplo: “*Quais franquias estão ativas no cadastro?*”). Constitui a entrada do experimento, e não o diálogo multi-turno com várias consultas sucessivas do utilizador.
- **Sessão:** episódio delimitado de interação do agente orquestrador para processar **uma** pergunta, desde a descoberta de metadados via *tools* MCP até a geração do SQL candidato e o encerramento da trilha registável. O orçamento de *tool calls* aplica-se por sessão, isto é, por pergunta tratada, com limite máximo de 6 chamadas.
- **Corrida:** execução experimental auditável da cadeia da pergunta → *tools* MCP → SQL final → validação → execução → registro, identificada por um *runId* único e arquivada em diretório dedicado de evidências. A corrida materializa a sessão no *harness* e é a unidade sobre a qual se calculam métricas e desfechos (*success*, *syntax_error*, entre outros).

Não se confunde **sessão** ou **corrida** com o processo servidor MCP em execução contínua no ambiente laboratorial: este permanece ativo entre perguntas, mas cada avaliação experimental reinicia o ciclo com identificador e registro próprios.

4.2.2 Arquitetura do artefato

A arquitetura metodológica do artefato segue uma organização por contextos e responsabilidades, aplicação e integrações. Em termos funcionais, o ciclo experimental é composto por seis passos: recepção da pergunta, descoberta de metadados, síntese de SQL, validação, execução controlada e registro de evidências. O painel (b) da Figura 3 sintetiza essa sequência e explicita a centralidade da descoberta de metadados no protocolo de geração de SQL.

Essa cadeia impõe uma restrição importante: o agente não deve produzir SQL sem antes consultar metadados canônicos via MCP. Portanto, a etapa de descoberta não é acessória; ela constitui condição de validade da corrida. Com isso, a geração deixa de ser avaliada apenas por fluência sintática e passa a ser avaliada pela aderência ao catálogo.

No nível de protocolo, são consideradas três *tools* mínimas de catálogo: **listagem de tabelas**, **descrição de tabela** e **listagem de relacionamentos**. O contrato MCP associado a essas *tools* define formato de resposta, taxonomia de erros de chamada e política de compatibilidade de versões. Para fins metodológicos, importa menos a estrutura JSON literal e mais o fato de que o protocolo torna observável quais metadados foram consultados antes da síntese de SQL.

A política de orçamento de *tool calls* é tratada como variável de controle experimental. Neste estudo, fixa-se orçamento de referência em 6 chamadas por sessão (equivalente a 6 chamadas por pergunta), como decisão de processo. O consumo efetivo é registrado em cada corrida para permitir comparações entre execuções.

A definição operacional de **erro estrutural** utilizada neste capítulo mantém-se alinhada ao escopo do trabalho: *considera-se erro estrutural quando a consulta referencia tabela, visão ou coluna ausente do catálogo canônico, quando usa schema incompatível com o catálogo ou quando constrói junções não suportadas pelos relacionamentos recuperados*. Essa definição orienta tanto a validação quanto a classificação de desfecho.

4.2.3 Procedimento de implementação

O procedimento é descrito como sequência metodológica de preparação do experimento, e não como roteiro de execução técnica. As etapas abaixo organizam o percurso mínimo necessário para garantir comparabilidade entre corridas:

1. **Delimitação do subconjunto de dados.** Seleciona-se o subconjunto de tabelas XPTO, fixa-se a versão do conjunto (*datasetVersion*) e define-se o escopo de cenários de pergunta.
2. **Preparação da base.** A massa é preparada no ambiente de dados e publicada de forma consistente para consulta SQL, preservando integridade referencial e controle de versão.
3. **Registro no catálogo canônico.** As entidades técnicas relevantes são registradas no Atlas para que as *tools* MCP reflitam o estado efetivo dos metadados disponíveis ao agente.

4. **Configuração do ciclo de inferência.** Define-se o provedor de modelo, versão do modelo, orçamento de *tools* e demais parâmetros de corrida em formato reprodutível.
5. **Execução controlada das corridas.** Cada pergunta percorre o fluxo completo de descoberta, síntese, validação e execução, com trilha observável de decisões.
6. **Consolidação de evidências.** Os artefatos de sessão são persistidos por *runId*, permitindo auditoria posterior e agregação de métricas.

Esse encadeamento garante rastreabilidade entre decisão metodológica e resultado observado. O detalhamento técnico de provisionamento de infraestrutura, integração de componentes e automações de carga é tratado no Capítulo de Desenvolvimento.

4.2.4 Protocolo de avaliação

A unidade de avaliação coincide com a **corrida** definida acima: cada execução completa da cadeia pergunta → *tools* MCP → SQL final → validação → execução → registro. O protocolo de execução por corrida é apresentado de forma padronizada:

1. inicialização da sessão e atribuição de *runId*;
2. consulta de metadados sob orçamento de chamadas;
3. geração de SQL candidato;
4. validação sintática;
5. validação estrutural contra catálogo canônico;
6. execução controlada sobre subconjunto XPTO;
7. registro da trilha e cálculo de métricas.

As métricas essenciais são: aderência estrutural ao catálogo, executabilidade e rastreabilidade da trilha pergunta, *tools*, metadados, SQL. Mantém-se também a métrica de orçamento de *tool calls*, por seu papel de custo/controle de busca. Como métricas secundárias, monitoram-se latência ponta a ponta e consumo de tokens quando disponíveis com coleta confiável.

A classificação de desfecho de cada corrida adota quatro classes mutuamente exclusivas: *success*, *syntax_error*, *structural_error* e *execution_error*. Tal classificação permite separar falhas de geração formal, falhas de aderência ao catálogo e falhas de execução no plano de dados.

Para cobertura mínima dos cenários definidos para o subconjunto XPTO, adota-se uma bateria piloto de 14 perguntas distribuídas em sete classes, com duas perguntas por classe. A Tabela 1 apresenta exemplos de formulações.

Tabela 1 – Bateria piloto de perguntas por cenário metodológico.

Cenário	Exemplos de pergunta (piloto)
Listagem simples	(i) Quais franquias estão ativas no cadastro? (ii) Liste os nomes e identificadores das franquias registradas.
Filtro por enum	(i) Quais projetos estão com status de execução? (ii) Quantos projetos estão arquivados?
Junção 1:N	(i) Quais unidades existem por franquias? (ii) Quais franquias possuem mais de uma unidade cadastrada?
Junção em cadeia	(i) Relacione franquias, segmento e rede correspondente. (ii) Quais segmentos concentram maior número de franquias por rede?
Agregação	(i) Qual o valor total de pagamentos por projeto? (ii) Quais projetos superam a média de pagamento registrada?
Filtro temporal	(i) Quantas transações ocorreram no último trimestre do período analisado? (ii) Qual a soma de transações por mês em janela temporal definida?
Junção geográfica	(i) Quais cidades por UF aparecem no subconjunto consultável? (ii) Quantos registros por UF estão associados às cidades catalogadas?

A bateria piloto tem finalidade metodológica de validação de fluxo e sensibilidade do protocolo. A ampliação do inventário de perguntas para corridas finais mantém o mesmo princípio de cobertura por cenário, com versionamento explícito em *datasetVersion* e rastreamento por *runId*.

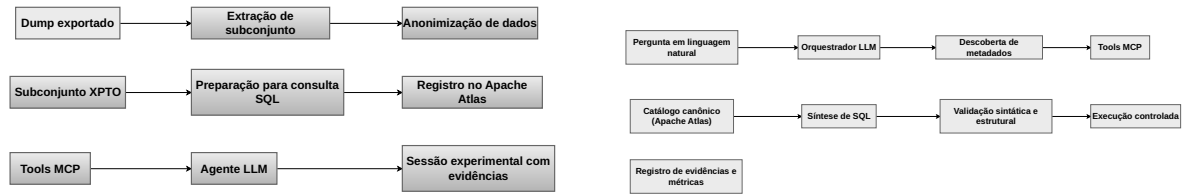
4.2.5 Registro experimental e ameaças à validade

A reprodutibilidade depende do registro padronizado de metadados e artefatos por execução. Cada corrida deve registrar, no mínimo, *runId*, *modelVersion*, *seed*, *commitHash*, *datasetVersion* e *contractsVersion*.

A estrutura de evidências por corrida adota diretório dedicado por execução (padrão *evidence/run-YYYYMMDD-HHMM-id/*). A segregação por *runId* evita sobrescrita de histórico e sustenta auditoria longitudinal das decisões experimentais.

No plano metodológico, a validade interna é tensionada por três fatores principais. O primeiro é o risco de sobreajuste da estratégia de prompt ao subconjunto de dados utilizado. O segundo é a variação temporal de comportamento dos modelos de linguagem entre janelas de execução. O terceiro é a simplificação inerente ao ambiente laboratorial, que não replica integralmente restrições de produção em ambiente não acadêmico.

Para mitigar esses riscos, adota-se: (i) cobertura de cenários heterogêneos na bateria de perguntas; (ii) versionamento explícito de dados e contratos; (iii) comparação entre provedores de modelo sob mesmos parâmetros de corrida; e (iv) registro completo da trilha pergunta–metadados–SQL. Em conjunto, tais medidas não eliminam as ameaças, mas tornam seus efeitos minimamente observáveis.



(a) Preparação e registro da massa XPTO.

(b) Fluxo do artefato *Text-to-SQL* com MCP.

Figura 3 – Fluxos metodológicos consolidados: preparação da massa de avaliação e ciclo experimental do artefato.

Em síntese, o capítulo metodológico estabelece um desenho experimental auditável para avaliação do artefato *Text-to-SQL*. A materialização técnica da arquitetura, da infraestrutura e das integrações é apresentada no Capítulo de Desenvolvimento, preservando neste capítulo a ênfase no método, nos critérios de avaliação e nas condições de reprodutibilidade.

5 Cronograma de execução

5.1 Cronograma de execução

Este capítulo consolida o **plano de execução** do trabalho no período de implementação experimental e redação, de **julho a novembro de 2026**, alinhado ao procedimento descrito no Capítulo 4. Os itens abaixo são **macro-atividades** editáveis: substitua rótulos, marcas na tabela e barras do Gantt conforme o andamento real do projeto.

5.1.1 Tabela mês × atividade

A Tabela 2 relaciona cada atividade aos meses em que está prevista. Use “×” para mês com trabalho previsto e “–” para mês sem atividade planejada (ou adapte a legenda).

Tabela 2 – Cronograma resumido por mês (modelo inicial; jul.–nov./2026).

Atividade	Jul.	Ago.	Set.	Out.	Nov.
Massa de teste, ambiente e reprodutibilidade	×	–	–	–	–
Fonte canônica de metadados (Atlas) e adaptador	×	×	–	–	–
Servidor MCP e ferramentas de consulta	–	×	×	–	–
Integração do agente, geração de SQL e validação	–	–	×	×	–
Corridas experimentais e consolidação de métricas	–	–	–	×	×
Redação e revisão dos capítulos da monografia	×	×	×	×	×
Figuras, referências bibliográficas e formatação final	–	–	×	×	×
Apresentação (slides) e preparação da defesa	–	–	–	–	×

Referências

- ANDROUTSOPOULOS, I.; RITCHIE, G. D.; THANISCH, P. *Natural Language Interfaces to Databases - An Introduction*. 1995. Disponível em: <<https://arxiv.org/abs/cmp-lg/9503016>>. Citado na página 10.
- ANTHROPIC; Model Context Protocol contributors. *Model Context Protocol*. 2024. Especificação aberta para integração de modelos de linguagem com ferramentas e contexto externo. Acesso em maio de 2026. Disponível em: <<https://modelcontextprotocol.io/>>. Citado na página 15.
- BAHDANAU, D.; CHO, K.; BENGIO, Y. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014. Citado nas páginas 11 e 13.
- FATHOLLAHZADEH, S.; MANSOUR, E.; BOEHM, M. Catdb: Data-catalog-guided, llm-based generation of data-centric ml pipelines. *Proceedings of the VLDB Endowment (PVLDB)*, v. 18, n. 8, p. 2639–2652, 2025. Disponível em: <<https://doi.org/10.14778/3742728.3742754>>. Citado nas páginas 14 e 17.
- JIAN, A. et al. Trust-sql: Tool-integrated multi-turn reinforcement learning for text-to-sql over unknown schemas. *arXiv preprint*, 2026. ArXiv:2603.16448v1. Citado nas páginas 12 e 16.
- LI, J. et al. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. In: OH, A. et al. (Ed.). *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2023. v. 36, p. 42330–42357. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2023/file/83fc8fab1710363050bbd1d4b8cc0021-Paper-Datasets_and_Benchmarks.pdf>. Citado nas páginas 11 e 17.
- LIU, M. et al. A systematic review of natural language interfaces for databases. *Frontiers of Computer Science*, Springer, v. 20, n. 11, p. 2011623, 2026. Citado nas páginas 8, 10, 11 e 13.
- POURREZA, M.; RAFIEI, D. *DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction*. 2023. Disponível em: <<https://arxiv.org/abs/2304.11015>>. Citado na página 11.
- RASCHKA, S. *Build a Large Language Model (from Scratch)*. Shelter Island, NY: Manning Publications, 2024. ISBN 978-1633437166. Citado nas páginas 4, 12 e 13.
- SUTSKEVER, I.; VINYALS, O.; LE, Q. V. Sequence to sequence learning with neural networks. *CoRR*, abs/1409.3215, 2014. Disponível em: <<http://arxiv.org/abs/1409.3215>>. Citado na página 10.
- The Apache Software Foundation. *Apache Atlas*. 2025. Documentação do catálogo de metadados e governança. Acesso em maio de 2026. Disponível em: <<https://atlas.apache.org/>>. Citado na página 15.
- VASWANI, A. et al. Attention is all you need. *CoRR*, abs/1706.03762, 2017. Disponível em: <<http://arxiv.org/abs/1706.03762>>. Citado nas páginas 12 e 13.
- WANG, Q. et al. Mci-sql: Text-to-sql with metadata-complete context and intermediate correction. *Proceedings of the VLDB Endowment (PVLDB)*, v. 14, n. 1, 2025. Citado nas páginas 12, 14 e 16.

YANG, B. et al. Hallucination detection for llm-based text-to-sql generation via two-stage metamorphic testing. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, v. 37, n. 6, 2025. ArXiv:2512.22250. Citado nas páginas 15 e 16.

YU, T. et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: RILOFF, E. et al. (Ed.). *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Brussels, Belgium: Association for Computational Linguistics, 2018. p. 3911–3921. Disponível em: <<https://aclanthology.org/D18-1425/>>. Citado na página 11.